

Paradigmes de Programmation Avancés

Étude Comparative : Tri Bitonique et Minage de Blockchain

NGUIYOU KEMMOE MERVEILLE PRECILIA
Matricule : 23U2478

Cours : INF342
Sous la direction du Dr Thomas MESSI NGUELE

17 mai 2026

Plan de la Présentation

- 1 Introduction Générale
- 2 Axe I : Le Réseau de Tri Bitonique
- 3 Axe II : Le Minage de Blockchain (Proof of Work)
- 4 Analyse Comparative
- 5 Conclusion

Contexte

L'évolution des architectures matérielles modernes exige la maîtrise de paradigmes de programmation fondamentalement distincts pour répondre aux défis de performance et de consensus distribué.

- **Processus étudiés** : Le **Tri Bitonique** (parallélisable) et le **Minage de Blockchain** (asymétrique).
- **Écosystèmes technologiques explorés** :
 - **Mercury** : Approche logico-fonctionnelle pure (déterminisme et modes).
 - **Scala** : Approche hybride orientée-objet et fonctionnelle sur la JVM.
 - **Q#** : Approche quantique native (Microsoft) basée sur les registres de qubits.

Definition (Séquence bitonique)

Une séquence est dite **bitonique** si elle croît de façon monotone puis décroît de la même manière, ou s'il existe une rotation cyclique vérifiant cette propriété.

Propriété clé

L'application de comparateurs à distance fixe $n/2$ divise la séquence en deux sous-séquences bitoniques indépendantes.

- **Complexité séquentielle** : $\mathcal{O}(n \log^2 n)$
- **Profondeur parallèle** : $\mathcal{O}(\log^2 n) \rightarrow$ Idéal pour architectures GPU, FPGA et processeurs quantiques.

Tri Bitonique : Approches Logique (Mercury) et Impérative (Scala)

Mercury (Décomposition Logique)

```
-: pred bitonic_sort(direction::in, list(int)::in, list(int)::out)
    is det.

bitonic_sort(_, [], []).
bitonic_sort(_, [X], [X]).
bitonic_sort(Dir, List, Sorted) :-
    Length = list.length(List), Half = Length / 2,
    split(List, Half, Left, Right),
    bitonic_sort(ascending, Left, SortedLeft),
    bitonic_sort(descending, Right, SortedRight),
    append(SortedLeft, SortedRight, BitonicSeq),
    bitonic_merge(Dir, BitonicSeq, Sorted).
```

Scala (Mutation In-Place sur Array)

```
private def bitonicSort[T](arr: Array[T], low: Int, cnt: Int,
    ascending: Boolean)(implicit ord: Ordering[T]): Unit = {
  if (cnt > 1) {
    val k = cnt / 2
    bitonicSort(arr, low, k, ascending = true)
    bitonicSort(arr, low + k, k, ascending = false)
    bitonicMerge(arr, low, cnt, ascending)
  }
}
```

Tri Bitonique : Approche Quantique (Q#)

Réseau de Tri Quantique par Portes Réversibles

```
namespace Quantum.Sorting {  
    open Microsoft.Quantum.Intrinsic;  
  
    operation ExecuteBitonicMerge(register : Qubit[], ascending : Bool) : Unit is Adj + Ctl {  
        let n = Length(register);  
        if (n > 1) {  
            let k = n / 2;  
            for i in 0 .. k - 1 {  
                Controlled SWAP([register[i]], (register[i], register[i + k]]));  
            }  
            ExecuteBitonicMerge(register[..k-1], ascending);  
            ExecuteBitonicMerge(register[k...], ascending);  
        }  
    }  
}
```

- Le tri repose sur des superpositions et des portes logiques quantiques conditionnelles (**Controlled SWAP**).

Axe II : Le Minage de Blockchain

Mécanisme Fondamental

Le minage par Preuve de Travail (*Proof of Work*) consiste à résoudre un défi mathématique asymétrique en cherchant une valeur itérative appelée **Nonce**.

Équation de Consensus

Trouver $n \in \mathbb{N}$ tel que $\text{SHA-256}(\text{Index} \parallel \text{PrevHash} \parallel \text{Timestamp} \parallel \text{Data} \parallel n) \leq T(\text{Difficulty})$

- **Asymétrie** : Très difficile et coûteux à calculer (Force brute), mais instantané à vérifier par le réseau.

Minage de Blockchain : Mercury et Scala

Mercury (Récursion Pure)

```
mine_block(Idx, Prev, Time, Data, Nonce, Target, Block) :-  
    Raw = string.format("%d|%s|%s|%s|%d",  
        [i(Idx), s(Prev), s(Time), s(Data), i(Nonce)]),  
    Hash = crypto.sha256(Raw),  
    ( if string.prefix(Hash, Target) then  
        Block = block(Idx, Prev, Time, Data, Nonce, Hash)  
    else  
        mine_block(Idx, Prev, Time, Data, Nonce + 1, Target, Block)  
    ).
```

Scala (Optimisation @tailrec)

```
@scala.annotation.tailrec  
def loop(nonce: Int): Block = {  
    val hash = calculateHash(index, prevHash, timestamp, data, nonce)  
    if (hash.startsWith(targetPrefix)) {  
        Block(index, prevHash, timestamp, data, nonce, hash)  
    } else {  
        loop(nonce + 1)  
    }  
}
```

Modélisation d'un Oracle pour l'Algorithme de Grover en Q#

```
namespace Quantum.Blockchain {
    open Microsoft.Quantum.Intrinsic;

    operation VerifyNonceOracle(nonceRegister : Qubit[], targetState : Qubit) : Unit is Adj + Ctl {
        // Inversion de phase quantique si le registre correspond au motif cible
        within {
            ApplyToEachA(X, nonceRegister[0 .. 2]);
        } apply {
            Controlled X(nonceRegister[0 .. 2], targetState);
        }
    }
}
```

- **Rupture technologique** : L'algorithme de Grover offre une accélération quadratique en $\mathcal{O}(\sqrt{N})$, réduisant théoriquement la complexité face à la force brute classique.

Analyse Comparative de la Performance des Paradigmes

Table – Matrice d'Évaluation Multicritères

Dimension d'Analyse	Mercury	Scala	Q#
Paradigme Principal	Logique-Fonctionnel Pur	Hybride Objet-Fonctionnel	Quantique Natif
Gestion de l'État	Immuabilité Absolue	Configurable (val / var)	Superposition & Intrication
Mécanisme de Tri	Décomposition Logique	Mutation In-Place ($\mathcal{O}(1)$)	Portes Unitaires Réversibles
Efficacité Minage	Récursion de queue pure	Parallélisme Threads JVM	Algorithme de Grover ($\mathcal{O}(\sqrt{N})$)

Indicateurs de Performance Observés (Simulation)

- **Scala** : Vitesse d'exécution brute critique grâce à la contiguïté mémoire des Array (Tri en 14.23 ms pour 65k éléments).
- **Mercury** : Sûreté transactionnelle mathématique élevée (zéro effet de bord).

Le choix d'un paradigme est intrinsèquement dicté par les contraintes industrielles et applicatives :

- **Mercury** s'impose pour les systèmes requérant une preuve formelle de non-régression et un contrôle strict de la logique logique/déterminisme.
- **Scala** s'avère optimal pour l'ingénierie logicielle d'entreprise, alliant performance d'exécution, parallélisme et robustesse de l'écosystème JVM.
- **Q#** préfigure les fondations de l'informatique de rupture, redéfinissant les barrières de la complexité algorithmique traditionnelle (accélération de Grover).

Merci pour votre aimable attention !